



Skill Workshop



Intro

- ✦ Skill engineering: één van de meest waardevolle AI-vaardigheden van 2026
- ✦ De meeste AI aanbieders ondersteunen een vorm van skills
- ✦ De discipline van ontwerpen, verbeteren en onderhouden van skills
- ✦ Skill engineering



Wat je leert

- ✦ Wat skills zijn en wanneer je ze inzet
- ✦ Een manier om je eigen skills te bouwen



Het probleem

- ✦ AI-modellen worden steeds krachtiger
- ✦ Maar blijven guard rails, context en instructies nodig hebben



Prompts vs. skills

- ✦ Prompt = eenmalige taak
- ✦ Skill = terugkerende taak met context en domeinkennis



Wanneer een prompt

- ✦ Je de taak zelden doet
- ✦ De instructie kort is
- ✦ Er geen vaste workflow is



Wanneer een skill

- ✦ Je hetzelfde proces herhaalt
- ✦ Consistente output belangrijk is
- ✦ Je het proces wilt kunnen verbeteren



Wat is een skill?

- ✦ Herbruikbare workflow voor een AI-model
- ✦ skill.md markdown bestand
- ✦ Verpakt in folder met reference files, voorbeelden van input/output, tools



Voor en door AI

- ✦ Skills kan je zelf schrijven...
- ✦ ...maar met AI gaat het makkelijker.



Simpel beginnen

- ✦ Begin met alleen een `skill.md`
- ✦ Voeg later referentiebestanden toe naarmate de skill volwassener wordt



Bezint eer ge begint

- ✦ De stap die meestal wordt overgeslagen
- ✦ Maar met grote impact



Vragen om over na te denken

- ✦ Hoe zou deze skill jouw werk beter maken?
- ✦ Welk proces wil je dat het volgt?
- ✦ Waar voegt menselijk oordeel waarde toe?
- ✦ Hoe ziet de gewenste output eruit?
- ✦ Wat kan er misgaan?
- ✦ Hoe test je of de skill goed werkt?



Eén duidelijk doel

- ✦ Veelgemaakte fout: één skill van 1500 regels voor een heel domein
- ✦ Geef een skill één duidelijk, afgebakend doel
- ✦ Twee/drie processen beschreven? Waarschijnlijk twee/drie skills



Het framework

- ✦ Geen exacte wetenschap, maar een solide startpunt



Frontmatter

- ✦ Name - de naam van de skill
- ✦ Description - wanneer de skill getriggerd wordt
- ✦ YAML Frontmatter



Goal

- ✦ Doel van de skill, kort gehouden
- ✦ Diepgang volgt in de processtappen



Inputs & context

- ✦ Welke informatie heeft het model nodig om de skill uit te kunnen voeren?
- ✦ Knowledge files, achtergrond informatie, assets
- ✦ LET OP: Wat het model al weet hoeft je niet te vertellen



Tools

- ✦ MCP
- ✦ sub-agents
- ✦ code execution



Process: de basis

- ✦ Proces stap voor stap uitleggen
- ✦ - actie
- ✦ - benodigde informatie
- ✦ - human-in-the-loop checkpoint



Human-in-the-loop

- ✦ Goed ontworpen skills ondersteunen human-in-the-loop momenten
- ✦ Punten waar het model pauzeert voor een vraag of goedkeuring
- ✦ Dit komt terug als checkpoint in de process-stappen



Checkpoint

- ✦ Vraag: vaak input veld of multi select
- ✦ Update: wat er gebeurt en waar je in het proces zit
- ✦ Zo houdt de gebruiker eigenaarschap



Laatste proces stap: controle

- ✦ Check of elke stap is doorlopen zoals bedoeld
- ✦ Check of alle benodigde informatie voorhanden is
- ✦ Ontbreekt er iets → meer vragen stellen



Output

- ✦ Echte voorbeelden toevoegen als example output file
- ✦ Tekst/creatief: meerdere variaties aanbieden
- ✦ Code/data: nadruk op validatie (testen/uitvoeren)



Rules

- ✦ Voorspel wat er mis kan gaan, schrijf het op als regel
- ✦ Leg ook vast wat je juist wél wilt zien
- ✦ Schrijf als opdracht: NIET, NOOIT, ALTIJD



Rules: verbod + alternatief

- ✦ Koppel een verbod aan het gewenste alternatief
- ✦ Voorbeeld: "NOOIT klantnamen noemen" + "ALTIJD sector noemen"
- ✦ Modellen volgen positieve instructies beter op



Rules: groeit continu

- ✦ Groeit: bij gebruik van de skill kan je bevindingen toevoegen
- ✦ Check per bevinding: losstaande rule, of hoort het ergens anders?
- ✦ Herhaling van terugkerende issues in andere vorm kan werken



Tips

- ✦ Houd de `skill.md` gefocust op het proces
- ✦ Al het andere hoort in referentiebestanden
- ✦ Vuistregel: `skill.md` onder de 500 regels



Stel dat je maar drie dingen onthoudt

- ✦ Geef elke skill één duidelijk, afgebakend doel – liever meerdere kleine skills dan één grote
- ✦ Houd de `skill.md` gefocust op alleen het proces – de rest hoort in een referentiebestand
- ✦ Bouw bewust human-in-the-loop checkpoints in

skill.md template



```
---
name: [naam]
description: [beschrijving – dit bepaalt of de skill wordt getriggerd]
---

## Goal
[Lopende tekst – één of twee zinnen die het doel van de skill beschrijven.]

## Inputs & context
[Welke knowledge files, documenten of assets nodig zijn.]

## Tools
[Welke MCP, sub-agents of code execution de skill gebruikt.]

## Process
1. [Stap 1: actie + benodigde informatie]
2. [Stap 2: actie] → [CHECKPOINT: wat moet de gebruiker hier controleren of
aanleveren?]
3. [Stap 3: update naar gebruiker over de status]
4. [Stap 4: actie] → [CHECKPOINT: ...]
5. Controleer of alle stappen zijn doorlopen zoals bedoeld en of alle benodigde
informatie voorhanden is. Ontbreekt er iets, stel dan eerst vragen voordat je
het resultaat oplevert.

## Output
[Hoe een goed resultaat eruitziet – verwijst naar een example output file indien
beschikbaar.]

## Rules
- ALTIJD [...]
- NOOIT [...]
```